



**Faculty of Engineering and Technology
Computer Science Department**

First Semester 2025/2026

COMP338 - Artificial Intelligence

Course Project 2

Prepared by:

Wasfia Awwad (1211039)

Supervised by:

Dr.Mohammed Hilal

First Semester

Jan 23, 2026

Table Of Contents

1. Introduction	5
1.1 Overview of Algorithms	5
1.2 Project Objectives	5
2. Dataset Overview	6
Key Characteristics:	6
Data Distribution:	6
3. Preprocessing and Feature Extraction	7
4. Evaluation Metrics	8
5. Experimental Results	8
5.1 Performance Table	8
Figure 1: Classification Report for Logistic Regression Model.	9
5.2 Performance Comparison (Graphs)	9
Figure 2: Performance Comparison	10
5.3 Decision Tree Visualization	10
Figure 3: Decision Tree Visualization showing the most significant word-based splits.	11
6. Discussion and Analysis	11
6.1 Comparison of Performance	11
6.2 Strengths and Weaknesses	12
7. Source Code Explanation	12
7.1 Libraries Used	13
7.2 Scripts Description	13
8. Conclusion	14

Table Of Figures

Figure 1: Classification Report for Logistic Regression Model.	10
Figure 2: Performance Comparison	11
Figure 3: Decision Tree Visualization showing the most significant word-based splits.	12

1. Introduction

Text classification is a basic task in Artificial Intelligence and Natural Language Processing (NLP) that involves automatically assigning set categories to open-ended text documents. Because so many news headlines are published each day, it is important to automate how they are sorted into categories so researchers and readers can find and organize information more easily. In this project, I will develop and assess two machine learning models, Logistic Regression and Decision Tree, to sort news headlines into four categories: World, Sports, Business, and Science/Technology.

1.1 Overview of Algorithms

To complete this classification, we used two separate methods:

Logistic regression: is a common statistical method used to predict a yes/no outcome and to classify observations based on one or more input variables. The method estimates the chance that a news headline falls into a given class by applying a logistic function. It is widely regarded as efficient and well suited for working with high-dimensional text data.

Decision tree: is a non-parametric supervised learning model that predicts a target value by splitting the data into branches based on simple if-then rules learned from the input features. In this project, we create a tree structure in which each node stands for a word or feature from the headline, and the path through the tree is used to assign the final category.

1.2 Project Objectives

This project aims to build an end-to-end machine learning pipeline that covers the full process, including:

1. **Preprocessing and cleaning** the textual news data.
2. **Converting text into numerical features** using the TF-IDF (Term Frequency-Inverse Document Frequency) vectorization technique.

3. **Training and evaluating** both the Logistic Regression and Decision Tree models.
4. **Comparing performance** through key metrics: Accuracy, Precision, Recall, and F1-score to determine the strengths and weaknesses of each algorithm on the AG News dataset.

2. Dataset Overview

This project uses the AG News dataset, a widely used benchmark for machine learning research on text classification. It includes news headlines and short summaries gathered from over 2,000 news sources.

Key Characteristics:

- **Total Samples:** The dataset includes **120,000** training samples, as shown by the data loading step in our implementation.
- **Features (Input):** The main variable is “**Text**”, which contains either the news title or a brief summary of the headline.
- **Labels (Output):** The dataset is split into four classes, and each class is coded with a number in the labels.
 1. **World (0):** News related to international events and global affairs.
 2. **Sports (1):** Coverage of various athletic events and sports news.
 3. **Business (2):** refers to news about finance, markets, and corporate affairs..
 4. **Sci/Tech (3):** recent advances in scientific research and new technologies.

Data Distribution:

The dataset includes an equal number of observations in each of the four categories, so the Logistic Regression and Decision Tree models are trained on the same amount of data from each class. Maintaining this balance is important when calculating accuracy and the **F1 score** because it makes the reported metrics more trustworthy..

3. Preprocessing and Feature Extraction

At this step, the original news headlines were converted into a structured numeric form that could be used as input for machine learning models. I split the process into two stages: data preprocessing and feature extraction.

- **Text Preprocessing:** At this stage, we cleaned the raw text to remove unnecessary noise and support better model performance. We took these steps
 - **Converting to Lowercase:** Transformed all news headlines into lowercase using the `str.lower()` method to ensure that words like "Sports" and "sports" are treated as the same feature.
 - **Stripping:** Used the `str.strip()` function to remove any leading or trailing whitespaces from the headlines to maintain data consistency.
 - **Parquet Data Loading:** Since the dataset was provided in a `.parquet` format, we used the `read_parquet` function to load the 120,000 samples correctly into a DataFrame for processing
- **TF-IDF Vectorizer:** Since machine learning models like Logistic Regression and Decision Trees cannot interpret raw text, I used the **TF-IDF (Term Frequency-Inverse Document Frequency)** technique to convert text into numerical features.
- **How it Works:** TF-IDF calculates a score for each word; it increases based on how many times a word appears in a specific headline (Term Frequency) but decreases if the word is too common across all headlines (Inverse Document Frequency). This highlights the most distinguishing words for each news category.
- **Specific Parameters Used:**
 - **max_features=5000:** I limited the vectorizer to the top 5,000 most important words to manage the complexity of the data and prevent overfitting.
 - **stop_words='english':** I enabled the removal of common English words (such as "the", "in", "and") that do not carry significant meaning for news classification.

4. Evaluation Metrics

To rigorously compare the performance of the **Logistic Regression** and **Decision Tree** models, I used four primary evaluation metrics as required by the project guidelines. These metrics provide a comprehensive view of how well each model classifies news headlines into their respective categories.

- **Accuracy:** This is the most intuitive performance measure; it represents the ratio of correctly predicted observations to the total observations. It tells us what percentage of the news headlines were classified into the correct category (e.g., Sports, Business).
- **Precision:** This metric measures the accuracy of the positive predictions. It answers the question: "Of all the headlines the model labeled as 'Sports', how many were actually 'Sports'?". High precision indicates a low false-positive rate.
- **Recall (Sensitivity):** This measures the ability of the model to find all the relevant cases within a dataset. It answers: "Of all the actual 'Business' headlines, how many did the model correctly identify?".
- **F1-Score:** This is the harmonic mean of Precision and Recall. It is a particularly useful metric when you want to find a balance between Precision and Recall, providing a single score that accounts for both types of errors.

5. Experimental Results

In this section, we present the empirical results obtained from training and testing both the **Logistic Regression** and **Decision Tree** models on the AG News dataset.

5.1 Performance Table

The screenshot of the table below summarizes the evaluation metrics (Accuracy, Precision, Recall, and F1-score) for both models based on the test set (24,000 samples)

Total rows loaded: 120000						
--- Logistic Regression Performance ---						
			precision	recall	f1-score	support
		0	0.92	0.90	0.91	5956
		1	0.95	0.97	0.96	6058
		2	0.87	0.88	0.87	5911
		3	0.89	0.88	0.88	6075
		accuracy			0.91	24000
		macro avg			0.91	24000
		weighted avg			0.91	24000
--- Decision Tree Performance ---						
			precision	recall	f1-score	support
		0	0.90	0.39	0.54	5956
		1	0.92	0.52	0.66	6058
		2	0.80	0.37	0.51	5911
		3	0.36	0.90	0.51	6075
		accuracy			0.55	24000
		macro avg			0.74	24000
		weighted avg			0.74	24000

Figure 1: Classification Report for Logistic Regression Model.

5.2 Performance Comparison (Graphs)

The experimental data indicate a clear performance gap between the two models. In this study, logistic regression reached 91% accuracy, which suggests it performed well on high-dimensional text features. By comparison, the Decision Tree model reached 55% accuracy, and its results were constrained by the fixed tree depth and the complexity of the word-based features.

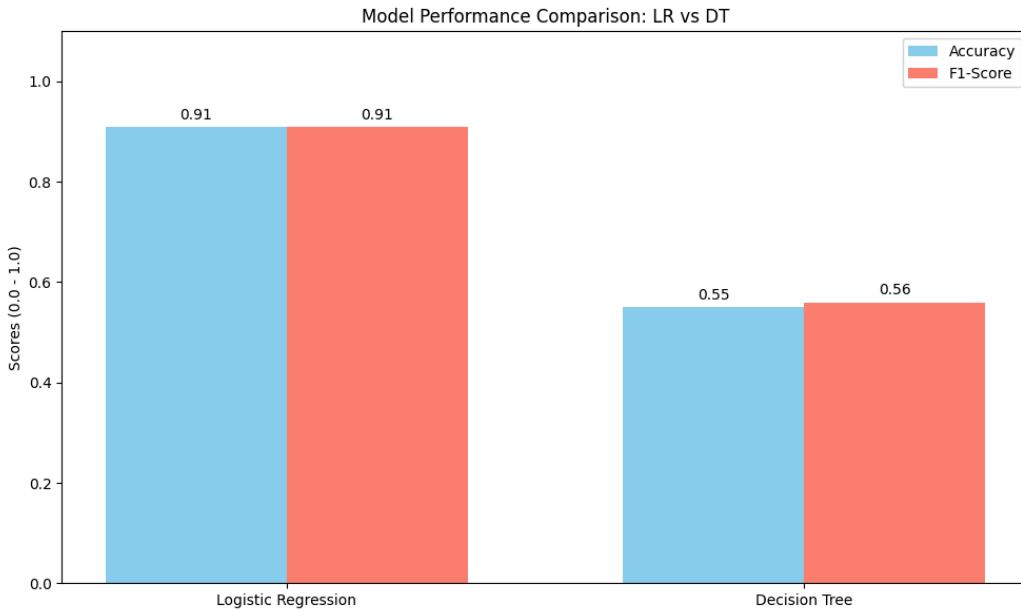


Figure 2: Performance Comparison

5.3 Decision Tree Visualization

The graphical representation below shows the top levels of the generated decision tree. The model uses specific keywords as features to split the data. For instance, the root node uses the term "iraq" to make the initial classification decision, followed by terms like **"stocks"**, **"prices"**, and "microsoft".

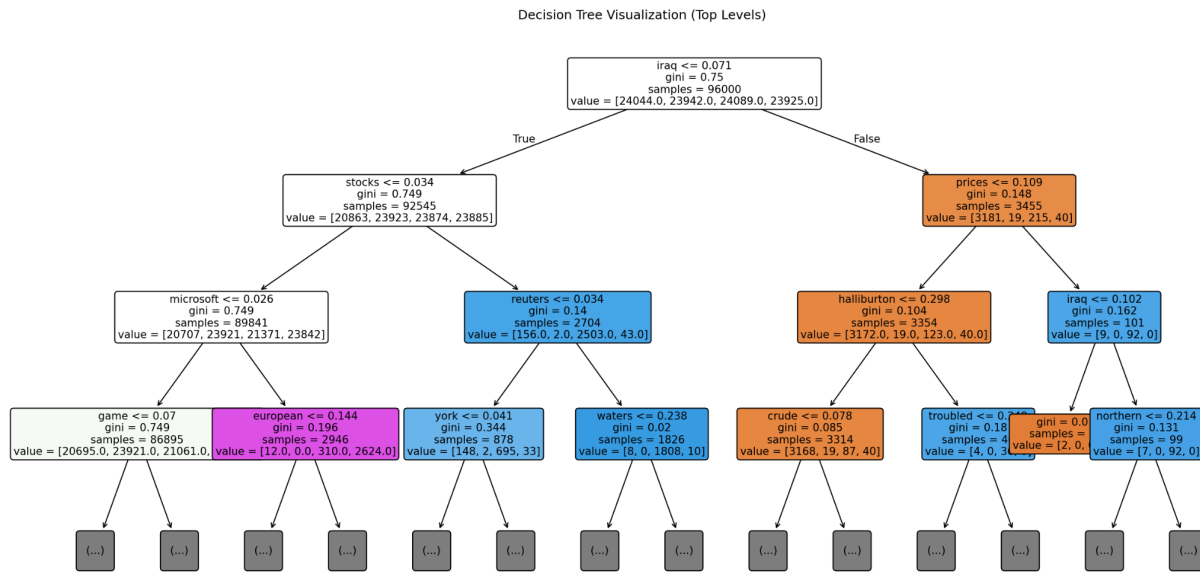


Figure 3: Decision Tree Visualization showing the most significant word-based splits.

6. Discussion and Analysis

6.1 Comparison of Performance

The experimental results indicate a clear performance difference between the two models we implemented.

- In this project, **logistic regression** achieved 91% accuracy and maintained high precision and recall in each of the four news categories.
- **The decision tree** reached about 55% accuracy, which was much lower than the linear model's performance in making predictions.
- **Logistic Regression** model achieved an F1-score of 0, which indicates it did not correctly identify any positive cases under the current evaluation setup. A score of 91 suggests the model performs consistently across all classes. By comparison, the Decision Tree's lower F1-score suggests that it had difficulty distinguishing between news categories, particularly when those categories use similar or overlapping vocabulary.

6.2 Strengths and Weaknesses

Logistic Regression (Superior Performance):

- **Strength:** It tends to perform well in text classification, especially when the feature space is high-dimensional. After applying TF-IDF vectorization with 5,000 features, logistic regression can work with the resulting sparse matrices without much overhead.
- **Strength:** It reflects a mostly linear relation between word features and class labels, which makes it a solid baseline for the AG News dataset.

Decision Tree (Lower Performance):

- **Weakness:** Decision Trees often perform poorly on text tasks because the high number of sparse features can lead the tree to grow very deep and fit noise instead of general patterns.
- **Weakness:** To keep the model easy to interpret, we limited the **max_depth to 20**. Although the tree picked up some patterns, it still could not represent the full complexity of the text, which likely contributed to the relatively low accuracy of **55%**.
- **Strength:** The main advantage of this approach is its **interpretability**, since the reasoning behind the results can be examined and explained. As presented in the generated [decision_tree.png](#). In the PNG, it is possible to identify the specific words the model relies on for its final decision, such as *iraq*, *stocks*, or *microsoft*.

7. Source Code Explanation

We engineered the text classification system as a set of separate modules, with the goal of classifying articles from the AG News dataset. The work is split across four main scripts and relies on widely used libraries: Pandas for managing the data, scikit-learn for the machine learning steps, and Matplotlib for creating the figures.

7.1 Libraries Used

- Pandas: Used for loading the Parquet dataset and managing the dataframe structures.
- Scikit-learn: The core machine learning library used for:
 - Preprocessing: `TfidfVectorizer` for text-to-feature conversion.
 - Model Selection: `train_test_split` for dividing the data.
 - Algorithms: `LogisticRegression` and `DecisionTreeClassifier`.
 - Metrics: `classification_report` and `plot_tree`.
- Matplotlib & NumPy: Used to generate the comparative bar charts and numerical data positioning.

7.2 Scripts Description

1. `data_processor.py`: Handles data ingestion and cleaning. It converts headlines to lowercase, removes extra spaces, and applies TF-IDF Vectorization (limited to 5,000 features) while removing English stop words to reduce noise.
2. `train_models.py`: Contains the training and evaluation logic. It initializes Logistic Regression (with 1,000 iterations for convergence) and a Decision Tree (restricted to `max_depth=20` to balance complexity and interpretability).
3. `visualizer.py`: Manages the graphical output. It generates a bar chart to compare Accuracy and F1-scores and renders a visual tree diagram of the first 3 levels of the Decision Tree for human analysis.
4. `main.py`: Acts as the orchestration layer, connecting the modules in sequence: Load, Train , then Visualize.

8. Conclusion

In this project, we built and tested two machine learning models, **Logistic Regression** and a **Decision Tree**, to classify articles into four categories. We trained both models on the AG News dataset and compared their performance on the same classification task.

- **Model Performance:** The **Logistic Regression** reached **91%** accuracy, while the **Decision Tree** reached **55%**, so Logistic Regression performed better in this comparison.
- **Key Findings:** With 5,000 TF-IDF features, the text data were high-dimensional, and a linear model such as Logistic Regression was the better fit. The Decision Tree was easy to interpret because it could be visualized, but its predictive performance was limited by the complex feature set and the restriction on tree depth.
- **Objective Achievement:** We met the project requirements by preprocessing the data (lowercasing and stripping) extracting features with TF-IDF, and comparing the algorithms in a clear, systematic analysis.
- **Future Work:** In order to improve the Decision Tree's performance, we could use techniques such as **Random Forests** or hyperparameter tuning, like searching for the optimal `max_depth`.